1.9 Numerical Methods

Mathematics — OCR A-Level (H240) — Paper 1: Pure Mathematics

Numerical methods are techniques for producing *approximate* answers when an exact closed-form solution is impossible, impractical, or unknown. Many equations and integrals that arise naturally in applied mathematics — from orbital mechanics, to finance, to machine learning — simply cannot be solved with the algebraic toolkit of A-level. For example, the transcendental equation $x = \cos x$ has a real root, but no combination of elementary functions expresses it; the integral $\int e^{-x^2}\,dx$ appears in statistics everywhere (normal distribution) but has no elementary antiderivative. In such cases we fall back on iterative procedures that generate sequences of better and better approximations, together with rigorous methods to bound the error. This chapter develops the three main root-finding techniques on the OCR H240 specification — change of sign, fixed-point iteration, and Newton-Raphson — and introduces numerical quadrature via the trapezium rule.

1. Why Numerical Methods?

Analytic algebra handles polynomial equations up to degree four and a small handful of other cases. Beyond that, most equations cannot be solved exactly in closed form. Classic examples:

- **Transcendental equations** mix polynomial and transcendental functions: $x = \cos x$, $e^x = 3x$, $\ln x = 1 - x$. No finite combination of radicals and elementary functions solves these.
- **Quintic and higher polynomials** are, in general, not solvable by radicals (Abel-Ruffini theorem, 1824).
- **Integrals without elementary antiderivatives**: $\int e^{-x^2}\,dx$, $\int \sin(x^2)\,dx$, $\int \frac{\sin x}{x}\,dx$ — all well-defined but no elementary closed form.

When exact methods fail, we design an algorithm that generates a sequence of approximations $x_0, x_1, x_2, \ldots$ converging to the true answer. Two kinds of error inevitably creep in:

Truncation error vs rounding error.

- *Truncation error* (sometimes called *method error*) is the error built into the algorithm itself — e.g. stopping a convergent sequence after finitely many steps, or approximating a curve by straight lines. It depends on the method and on how many iterations / strips you use.
- *Rounding error* is the error introduced by representing numbers with finitely many digits (e.g. a calculator keeping 10 significant figures). It accumulates with every arithmetic operation.

Both must be kept under control. A good numerical method reduces truncation error quickly enough that rounding error stays dominated by it.

Exam tip — state the degree of accuracy. Every numerical answer in an OCR exam should carry an explicit statement of accuracy: "to 3 decimal places", "to 3 significant figures", or "correct to 4 d.p.". Marks are routinely lost by quoting an answer to too few or too many figures, or by failing to say which. If the question specifies the accuracy, round at the very end — never mid-calculation — and keep at least one extra guard digit throughout the working.

2. Locating Roots — The Change of Sign Method

The simplest way to show that an equation $f(x) = 0$ has a root in an interval is to use the *Intermediate Value Theorem*.

Root of an equation. A *root* of $f(x) = 0$ is a value α such that $f(\alpha) = 0$. Graphically, it is a value of x where the curve $y = f(x)$ crosses (or touches) the x-axis.

Change of sign (Intermediate Value Theorem). If f is *continuous* on the closed interval $[a, b]$ and $f(a)$ and $f(b)$ have opposite signs (that is, $f(a) \cdot f(b) < 0$), then there exists at least one root $\alpha \in (a, b)$ with $f(\alpha) = 0$.

The reasoning is geometric: a continuous curve cannot jump from a positive value at $x = a$ to a negative value at $x = b$ without passing through $y = 0$ somewhere in between. The theorem is an *existence* statement only — it guarantees at least one root, not a unique one, and says nothing about its exact location.

Worked Example 1 — Locate a root of $f(x) = x^3 - 2x - 5$

Show that the equation $x^3 - 2x - 5 = 0$ has a root between $x = 2$ and $x = 3$.

```
f(x) = x³ − 2x − 5 is a polynomial, therefore continuous on ℝ.
Evaluate at the endpoints:
   f(2) = 8  − 4 − 5 = −1     (negative).
   f(3) = 27 − 6 − 5 = 16     (positive).
Since f(2) < 0 and f(3) > 0, there is a change of sign.
By the Intermediate Value Theorem, there is at least one root
α ∈ (2, 3) with f(α) = 0.                                   ■

// This equation was used by Wallis and Newton in the 1600s to introduce
// iterative methods — the "root between 2 and 3" is α ≈ 2.0946.
```

Exam tip — always quote f(a) and f(b) and the word "continuous". A fully-marked change-of-sign argument has four parts: (i) state that f is continuous on $[a, b]$; (ii) compute $f(a)$ with sign; (iii) compute $f(b)$ with sign; (iv) conclude "since there is a change of sign, there is a root in (a, b)". Skipping any of these — particularly the continuity clause — costs method marks.

Two important cautions apply to change-of-sign arguments:

- **Continuity is essential.** The function $f(x) = 1/x$ satisfies $f(-1) = -1 < 0$ and $f(1) = 1 > 0$, but has no root in $(-1, 1)$ — because it is discontinuous at $x = 0$, where it diverges to $\pm\infty$. The change of sign happens across a vertical asymptote, not through the axis.
- **Sign change detects only an odd number of roots.** If $f(a)$ and $f(b)$ have the *same* sign, there could still be zero, two, four, or any even number of roots in (a, b). A change of sign finds at least one, but not the total count.

Worked Example 2 — Missing a double root

```
Consider f(x) = (x − 1)² on the interval [0, 2].
f(0) = 1 > 0;  f(2) = 1 > 0.   No sign change.
Yet f(1) = 0, so there IS a root at x = 1.
```

```
4.  Conclusion: change-of-sign is sufficient but not necessary for a root.
5.  Touching the axis (repeated root) is invisible to sign-change tests.
```

3. Interval Bisection

Once a sign change locates a root in $[a, b]$, we can narrow down its position by repeatedly halving the interval — the *bisection method*.

Interval bisection algorithm. Given a continuous f and an interval $[a, b]$ with $f(a) \cdot f(b) < 0$:

1. Compute the midpoint $c = (a + b)/2$.
2. Evaluate $f(c)$. If $f(c) = 0$, the root is c; stop.
3. If $f(a) \cdot f(c) < 0$, the sign change is in $[a, c]$; set $b \leftarrow c$.
4. Otherwise the sign change is in $[c, b]$; set $a \leftarrow c$.
5. Repeat until the interval width $b - a$ is below the required tolerance.

Because each pass halves the bracket width, after n iterations the remaining interval has width

$$w_n = \frac{b - a}{2^n},$$

and the root α lies within $w_n/2$ of the current midpoint. This gives a guaranteed error bound — a rare luxury among numerical methods. Convergence is *linear*: each step wins one bit of accuracy, about 0.3 of a decimal digit per iteration. It is steady but slow.

Worked Example 3 — Three bisection steps on $f(x) = x^3 - 2x - 5$ in [2, 3]

```
1.  Start with a = 2, b = 3.  f(2) = -1, f(3) = 16.
2.  Step 1: c₁ = (2 + 3)/2 = 2.5.
3.     f(2.5) = 15.625 - 5 - 5 = 5.625 (positive).
4.     f(2) < 0, f(2.5) > 0  ⇒ sign change in [2, 2.5].  b ← 2.5.
5.  Step 2: c₂ = (2 + 2.5)/2 = 2.25.
6.     f(2.25) = 11.390625 - 4.5 - 5 = 1.890625 (positive).
```

```
    f(2) < 0, f(2.25) > 0  ⇒ sign change in [2, 2.25]. b ← 2.25.
Step 3: c₃ = (2 + 2.25)/2 = 2.125.
    f(2.125) = 9.5957 ... - 4.25 - 5 = 0.3457 ... (positive).
    f(2) < 0, f(2.125) > 0 ⇒ sign change in [2, 2.125]. b ← 2.125.

After 3 iterations the root is known to lie in [2, 2.125],
width = 0.125, midpoint estimate α ≈ 2.0625 ± 0.0625.
(True root is α ≈ 2.0946 — consistent with the bracket.)

// After n iterations the bracket width is (b - a)/2ⁿ = 1/2ⁿ.
// To reach 3 decimal places (error < 0.0005) need 2ⁿ > 2000, so n ≥ 11.
```

Iteration n	Left a	Right b	Midpoint c	$f(c)$	Keep half
0	2.0000	3.0000	2.5000	+5.6250	[2.0, 2.5]
1	2.0000	2.5000	2.2500	+1.8906	[2.0, 2.25]
2	2.0000	2.2500	2.1250	+0.3457	[2.0, 2.125]
3	2.0000	2.1250	2.0625	-0.3513	[2.0625, 2.125]
4	2.0625	2.1250	2.0938	-0.0089	[2.0938, 2.125]
5	2.0938	2.1250	2.1094	+0.1668	[2.0938, 2.1094]

Exam tip — state the sign of every evaluation. In bisection work, the examiner needs to see that you have checked the sign of f at each new midpoint and correctly decided which half retains the sign change. Writing "$f(2.5) = 5.625 > 0$" (with the inequality) earns both the computation mark and the decision mark in one line.

4. Fixed-Point Iteration

A second family of root-finding techniques works by rewriting $f(x) = 0$ in the form $x = g(x)$ — so that a root α of f is a *fixed point* of g, meaning $g(\alpha) = \alpha$. We then iterate:

$$x_{n+1} = g(x_n), \qquad n = 0, 1, 2, \ldots$$

starting from some initial guess x_0. If the sequence $\{x_n\}$ converges, its limit must be a fixed point of g (and therefore a root of f).

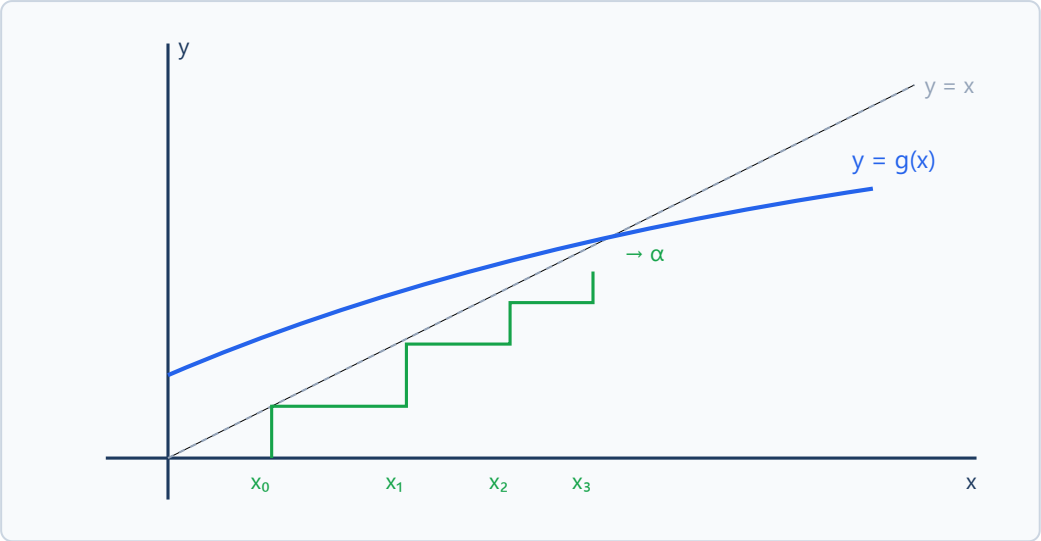
Fixed-point iteration. To solve $f(x) = 0$ by fixed-point iteration:

1. Rearrange the equation into the form $x = g(x)$. There are usually several ways to do so.
2. Choose a starting value x_0 close to the suspected root.
3. Compute successive terms $x_{n+1} = g(x_n)$ until the difference between consecutive values is below a chosen tolerance.

Convergence criterion. Let α be a fixed point of g (so $g(\alpha) = \alpha$), and suppose g is differentiable near α. The iteration $x_{n+1} = g(x_n)$ converges locally to α if and only if \[|g'(\alpha)| < 1. \] If $|g'(\alpha)| > 1$, the iteration diverges away from α no matter how good the starting guess (unless you happen to start exactly on α). The case $|g'(\alpha)| = 1$ is borderline.

Visually, the iteration corresponds to bouncing between the line $y = x$ and the curve $y = g(x)$:

- **Staircase diagram**: when $0 < g'(\alpha) < 1$, successive iterates approach α monotonically from one side, tracing a staircase pattern.
- **Cobweb diagram**: when $-1 < g'(\alpha) < 0$, iterates alternate about α, producing a spiral (cobweb) pattern that closes in on the fixed point.
- **Divergence**: if $|g'(\alpha)| > 1$, the staircase or cobweb walks outward, running away from α.



Worked Example 4 — Solve $x = \cos x$ by fixed-point iteration

```
Here g(x) = cos x. Take x₀ = 0.5 (in radians).
g'(x) = −sin x, and at the root α ≈ 0.739 we have
   |g'(α)| = |sin 0.739| ≈ 0.674 < 1.
So the iteration converges (with alternating signs — cobweb pattern).

n   xₙ          xₙ₊₁ = cos xₙ
0   0.50000     cos(0.5)       = 0.87758
1   0.87758     cos(0.87758)   = 0.63901
2   0.63901     cos(0.63901)   = 0.80269
3   0.80269     cos(0.80269)   = 0.69478
4   0.69478     cos(0.69478)   = 0.76820
5   0.76820     cos(0.76820)   = 0.71917
6   0.71917     cos(0.71917)   = 0.75236
 ...
After ~25 iterations: α ≈ 0.73909  (to 5 d.p.).

// Because g'(α) is negative, the iterates ALTERNATE above and below α —
// classic cobweb. Convergence is linear with rate ≈ 0.674 per step.
```

Worked Example 5 — A divergent rearrangement

Consider solving $x^3 - 2x - 5 = 0$. One rearrangement is $x = (x^3 - 5)/2$. Show this iteration diverges near the root $\alpha \approx 2.09$.

```
g(x) = (x³ − 5)/2.
g'(x) = 3x²/2.
At α ≈ 2.09: |g'(α)| ≈ 3(2.09)²/2 ≈ 6.55.
Since |g'(α)| > 1, the iteration DIVERGES.

Confirm: x₀ = 2.
   x₁ = (2³ − 5)/2 = 1.5.
   x₂ = (1.5³ − 5)/2 = (3.375 − 5)/2 = −0.8125.
   x₃ = (−0.8125³ − 5)/2 = (−0.5364 − 5)/2 = −2.7682.
   x₄ = (−2.7682³ − 5)/2 ≈ (−21.23 − 5)/2 ≈ −13.1  (!!)
Iterates fly off to −∞. The rearrangement is USELESS for this root.

A better rearrangement: x = (2x + 5)^(1/3). Then
   g'(x) = (2/3)(2x + 5)^(−2/3); at α ≈ 2.09  |g'(α)| ≈ 0.152 < 1.  ✓
```

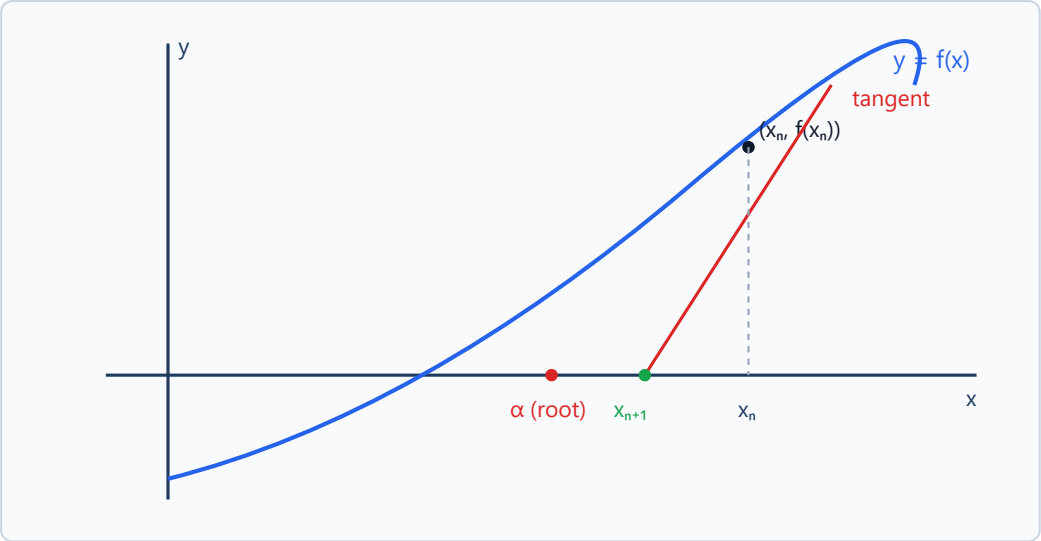
Exam tip — check $|g'(\alpha)| < 1$ before iterating. A common OCR question gives two or three rearrangements of the same equation and asks which one converges. The answer is always the one whose $|g'|$ is smallest at the root. Differentiate, substitute the approximate root, and report the absolute value — state whether it is less than or greater than 1, and conclude convergent/divergent.

5. Newton-Raphson Method

Newton-Raphson is a powerful, fast-converging root finder that uses the local tangent line to the curve $y = f(x)$ at the current estimate.

Newton-Raphson iteration. Given a differentiable f and a starting estimate x_0 near a root α, \[x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}. \] Geometrically, the next estimate x_{n+1} is the x-intercept of the tangent line to $y = f(x)$ at the point $(x_n, f(x_n))$.

Derivation: the tangent at x_n has equation $y - f(x_n) = f'(x_n)(x - x_n)$. Setting $y = 0$ and solving for x gives the intercept $x = x_n - f(x_n)/f'(x_n)$ — which we call x_{n+1}.



Quadratic convergence. Near a *simple* root α (where $f'(\alpha) \neq 0$), Newton-Raphson converges *quadratically*: the error $e_n = x_n - \alpha$ satisfies $|e_{n+1}| \approx K|e_n|^2$ for some constant K. Practically, this means the number of correct decimal places roughly *doubles* with each iteration once you are close enough.

Worked Example 6 — Approximate $\sqrt{7}$ by solving $x^2 - 7 = 0$

```
f(x) = x² - 7,   f'(x) = 2x.
Newton-Raphson:
   x_(n+1) = x_n - (x_n² - 7)/(2x_n)
           = (x_n + 7/x_n) / 2.            (the "Babylonian" formula)
Start x₀ = 3 (since 3² = 9 is close to 7).

n   x_n             x_(n+1) = (x_n + 7/x_n)/2
0   3.00000000      (3 + 7/3)/2       = (3 + 2.3333)/2    = 2.66667
1   2.66666667      (2.6667 + 2.625)/2                     = 2.64583
2   2.64583333      (2.6458 + 2.64567)/2                   = 2.64575
3   2.64575131      (2.64575131 + 2.64575131)/2            = 2.64575131

Converged to x ≈ 2.6457513 (which equals √7 to 7 d.p.).
Notice how the accuracy roughly doubles each step — quadratic convergence.
```

```
// This is the 4000-year-old "Babylonian method" for square roots –
// but it is exactly Newton-Raphson applied to x² – N = 0.
```

5.1 When Newton-Raphson fails

Despite its speed, Newton-Raphson is not foolproof. It can fail or misbehave in several situations:

- **Zero derivative ($f'(x_n) = 0$):** the formula divides by zero. Geometrically, the tangent at x_n is horizontal, so it never meets the x-axis. Choose a different starting value.
- **Poor initial guess:** far from α, the tangent line's x-intercept can be far worse than x_n, and subsequent iterates can run away to infinity or hop to a different root.
- **Inflection points:** if f'' changes sign near the root, the iteration may oscillate.
- **Repeated roots:** if α is a double root ($f'(\alpha) = 0$), quadratic convergence is lost and the method becomes only linearly convergent.

Worked Example 7 — A starting value that fails

```
1.  Apply Newton-Raphson to f(x) = x³ – 2x + 2 with x₀ = 0.
2.  f'(x) = 3x² – 2.
3.  x₁ = 0 – f(0)/f'(0) = 0 – 2/(–2) = 1.
4.  f(1) = 1 – 2 + 2 = 1;   f'(1) = 3 – 2 = 1.
5.  x₂ = 1 – 1/1 = 0.
6.  x₃ = 0 – (2)/(–2) = 1.
7.  x₄ = 1 – 1 = 0.
8.  The iteration CYCLES between 0 and 1 – it never converges.
9.  The actual root is near α ≈ –1.7693, far from the cycle.

// Lesson: even a well-behaved cubic can trap Newton-Raphson in a 2-cycle.
// A sketch of y = f(x) BEFORE choosing x₀ is always a good idea.
```

Exam tip — Newton-Raphson can diverge if the starting value is poor. Questions often ask "explain why this iteration fails when $x_0 = \ldots$". The standard answers are: (i) $f'(x_0) = 0$, so the tangent is horizontal and x_1 is undefined; (ii) the tangent at x_0 misses the neighbourhood of the

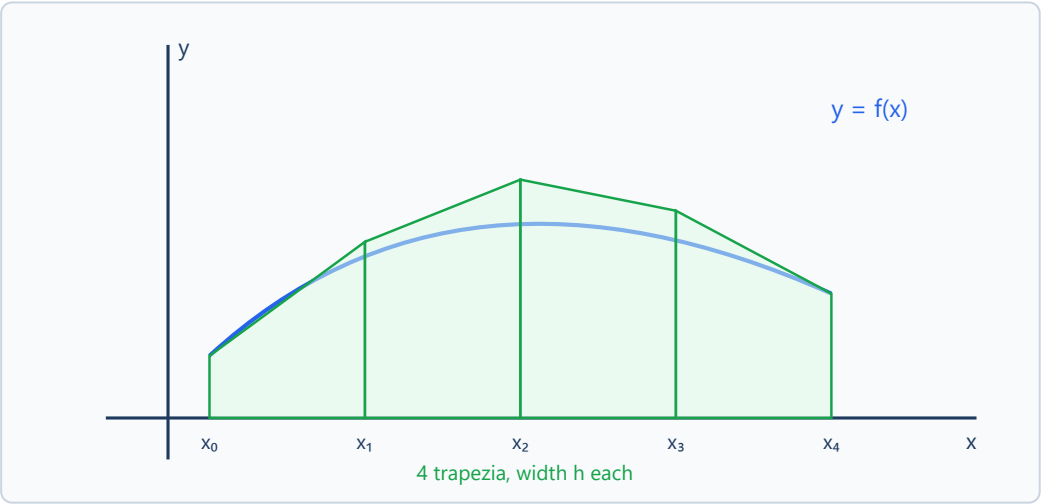
root entirely; (iii) the starting value is on the "wrong" side of a turning point. Always sketch $y = f(x)$ before applying the method.

6. The Trapezium Rule

Moving from roots to integrals: when an antiderivative is unavailable (or prohibitively hard), we can still estimate the definite integral $\int_a^b f(x)\,dx$ by approximating the area under the curve with a collection of trapezia.

Trapezium rule. Divide $[a, b]$ into n strips of equal width $h = (b - a)/n$. Label the endpoints $x_0 = a, x_1 = a + h, \ldots, x_n = b$ and let $y_i = f(x_i)$. Then \[\int_a^b f(x)\, dx \;\approx\; \frac{h}{2}\Big[y_0 + 2(y_1 + y_2 + \cdots + y_{n-1}) + y_n\Big]. \] The end ordinates y_0 and y_n are counted once; every interior ordinate is counted twice.

The formula comes from approximating each strip by a trapezium of parallel sides y_i and y_{i+1} and height h. The area of one trapezium is $\frac{h}{2}(y_i + y_{i+1})$; summing and collecting repeated terms gives the formula above.



Worked Example 8 — Approximate $\int_0^1 \frac{1}{1+x^2}\,dx$ with 4 strips

The exact value is $[\arctan x]_0^1 = \pi/4 \approx 0.78540$.

```
n = 4, a = 0, b = 1, so h = 1/4 = 0.25.
Tabulate y_i = 1/(1 + x_i²):
   x₀ = 0.00  →  y₀ = 1/1       = 1.00000
   x₁ = 0.25  →  y₁ = 1/1.0625  = 0.94118
   x₂ = 0.50  →  y₂ = 1/1.25    = 0.80000
   x₃ = 0.75  →  y₃ = 1/1.5625  = 0.64000
   x₄ = 1.00  →  y₄ = 1/2       = 0.50000

Apply the formula:
   I ≈ (h/2) [y₀ + 2(y₁ + y₂ + y₃) + y₄]
     = (0.25/2) [1.00000 + 2(0.94118 + 0.80000 + 0.64000) + 0.50000]
     = 0.125   [1.00000 + 2(2.38118) + 0.50000]
     = 0.125   [1.00000 + 4.76236 + 0.50000]
     = 0.125 × 6.26236
     ≈ 0.78280.

Compare with exact π/4 ≈ 0.78540  →  error ≈ 0.00260  (about 0.3 %).

// The integrand is concave on [0, 1], so the trapezia lie BELOW the curve,
// giving an under-estimate of the true area. This matches our 0.78280 < 0.78540.
```

6.1 Error behaviour of the trapezium rule

The error in the trapezium rule shrinks as the number of strips increases:

Trapezium rule — error. For a twice-differentiable integrand, the total error E_n in approximating $\int_a^b f(x)\,dx$ with n equal strips satisfies \[E_n = -\frac{(b - a)^3}{12 n^2}\, f''(\xi) \quad \text{for some } \xi \in (a, b). \] In particular $E_n = O(h^2)$, so doubling the number of strips divides the error by roughly 4.

Strips n	Estimate of $\pi/4$	Error	Ratio to previous
2	0.77500	0.01040	—
4	0.78280	0.00260	$\approx 1/4$
8	0.78475	0.00065	$\approx 1/4$

16	0.78524	0.00016	$\approx 1/4$
32	0.78536	0.00004	$\approx 1/4$

The fourfold improvement per doubling is the hallmark of an $O(h^2)$ method.

6.2 Over- vs under-estimate (concavity)

The trapezia used in the rule are straight-line chords between consecutive points on the curve. Whether they lie above or below the true curve depends on the concavity of f on the strip:

- If $f''(x) > 0$ (curve is convex / "cup up"), each chord lies *above* the curve — the trapezium rule **over-estimates** the integral.
- If $f''(x) < 0$ (curve is concave / "cap down"), each chord lies *below* the curve — the trapezium rule **under-estimates**.
- If f is linear ($f'' = 0$) the rule is exact.

Exam tip — state the number of strips (or ordinates) and h. Whenever you apply the trapezium rule, write down: (i) the number of strips n (beware — n strips means $n+1$ ordinates); (ii) the strip width $h = (b-a)/n$; (iii) a neat table of x_i and y_i values; (iv) the substitution into the formula. OCR examiners report that missed marks most often come from confusing "strips" with "ordinates", or from dropping an interior term.

7. Simpson's Rule (Brief Mention)

Beyond the trapezium rule lies *Simpson's rule*, which fits a parabola (quadratic) through every three consecutive ordinates rather than a straight line. The formula (for an *even* number n of strips) is

$$\int_a^b f(x)\,dx \approx \frac{h}{3}\Big[y_0 + 4(y_1 + y_3 + \cdots) + 2(y_2 + y_4 + \cdots) + y_n\Big].$$

It is much more accurate than the trapezium rule (error is $O(h^4)$ rather than $O(h^2)$), so doubling the strips divides the error by 16 instead of 4. **Simpson's rule is not on the core OCR H240 specification** — it sits on some other specifications (MEI, Edexcel Further) — but you may meet it informally. For standard OCR A-level pure, the trapezium rule is the only quadrature method you are expected to apply.

8. Error Analysis and Convergence

Every numerical algorithm generates a sequence of approximations; the key questions are *how fast* and *how accurately*.

Order of convergence. A sequence $\{x_n\} \to \alpha$ is said to converge with order p if there is a constant $K > 0$ with \[|x_{n+1} - \alpha| \leq K\, |x_n - \alpha|^p \] for all sufficiently large n.

- $p = 1$ — *linear* convergence. Each step multiplies the error by a fixed constant less than 1 (bisection, fixed-point iteration).
- $p = 2$ — *quadratic* convergence. The number of correct digits roughly doubles per step (Newton-Raphson at a simple root).

In practice, you cannot know α exactly — so you cannot measure the error directly. A standard proxy is the gap between successive iterates: iterate until

$$|x_{n+1} - x_n| < \varepsilon$$

for a chosen tolerance ε. For a rapidly convergent method this is a safe indicator of accuracy; for a slowly convergent one it may overstate progress. A more reliable check for a root-finding problem is to evaluate $f(x_n)$ — if $|f(x_n)|$ is small and $|f'(x_n)|$ is not tiny, then x_n is close to a root.

Exam tip — include units and the degree of accuracy. When a numerical method produces an answer with physical meaning (velocity in m s^{-1}, area in m^2, etc.), always write the units on the final line. For pure-number answers, state the accuracy: "to 3 decimal places" or "correct to 4 significant figures". Keep one extra guard digit in the working, then round only at the very end.

9. Choice of Method

Method	Speed (order of convergence)	Reliability	Prerequisites	Best when...

Change of sign / bisection	Linear, very slow (1 bit per step)	Extremely robust — never diverges if started with a sign change	f continuous, known sign change	You need a guaranteed bracket and don't mind the slow rate.
Fixed-point iteration $x = g(x)$	Linear (rate $\lvert g'(\alpha)\rvert$)	Works only if $\lvert g'(\alpha)\rvert < 1$; divergent otherwise	A good rearrangement of $f(x) = 0$	A neat rearrangement exists and you don't need quadratic speed.
Newton-Raphson	Quadratic (very fast near a simple root)	Can diverge, cycle, or hit $f'(x_n) = 0$; needs a decent start	f differentiable, f' easy to evaluate	You need high accuracy quickly and the derivative is cheap.
Trapezium rule	$O(h^2)$ — halving h quarters the error	Always returns an answer; accuracy bounded by concavity of f	Integrand tabulated at equally-spaced points	Integrating by hand when no antiderivative is available.

10. Applied Worked Examples

Worked Example 9 — Bisection to 3 d.p. for $x^3 - 2x - 5 = 0$

```
f(x) = x³ − 2x − 5;  f(2) = −1, f(3) = 16  ⇒ root in (2, 3).
Bisect until the bracket width < 0.0005 (half of 10⁻³).

n   a         b         c=(a+b)/2   f(c)          keep
0   2.00000   3.00000   2.50000     +5.6250       [2, 2.5]
1   2.00000   2.50000   2.25000     +1.8906       [2, 2.25]
2   2.00000   2.25000   2.12500     +0.3457       [2, 2.125]
3   2.00000   2.12500   2.06250     −0.3513       [2.0625, 2.125]
4   2.06250   2.12500   2.09375     −0.0089       [2.0938, 2.125]
5   2.09375   2.12500   2.10938     +0.1668       [2.0938, 2.1094]
6   2.09375   2.10938   2.10156     +0.0785       [2.0938, 2.1016]
7   2.09375   2.10156   2.09766     +0.0347       [2.0938, 2.0977]
```

```
8   2.09375   2.09766   2.09570     +0.0129         [2.0938, 2.0957]
9   2.09375   2.09570   2.09473     +0.0020         [2.0938, 2.0947]
10  2.09375   2.09473   2.09424     −0.0034         [2.0942, 2.0947]
11  2.09424   2.09473   2.09448     −0.0007         [2.0945, 2.0947]

Bracket width = 0.00025 < 0.0005, so we stop.
Root α ≈ 2.094 (to 3 d.p.).
```

Worked Example 10 — Fixed-point iteration $x = (2x+5)^{1/3}$ from $x_0 = 2$

```
Rearrange x³ − 2x − 5 = 0 as x³ = 2x + 5, so x = (2x + 5)^(1/3).
g(x) = (2x + 5)^(1/3);  g'(x) = (2/3)(2x + 5)^(−2/3).
At α ≈ 2.095:  g'(α) ≈ (2/3)(9.19)^(−2/3) ≈ 0.152 < 1.  ✓ CONVERGES.

n     x_n              x_(n+1) = (2x_n + 5)^(1/3)
0     2.00000000       (9)^(1/3)          = 2.08008
1     2.08008382       (9.16017)^(1/3)    = 2.09235
2     2.09235450       (9.18471)^(1/3)    = 2.09421
3     2.09421117       (9.18842)^(1/3)    = 2.09450
4     2.09449430       (9.18899)^(1/3)    = 2.09454
5     2.09453743       (9.18907)^(1/3)    = 2.09455
6     2.09454399       (9.18909)^(1/3)    = 2.09455

Converged: α ≈ 2.0946 (to 4 d.p.).
Note the linear convergence rate ≈ 0.15 per step − consistent with |g'(α)|.
```

Worked Example 11 — Newton-Raphson on $x^2 = 7$, first three iterations

```
f(x) = x² − 7,  f'(x) = 2x.
Iteration: x_(n+1) = x_n − (x_n² − 7)/(2 x_n).
x₀ = 3.

x₁ = 3 − (9 − 7)/6       = 3 − 2/6       = 3 − 0.33333 = 2.66667
x₂ = 2.66667 − (7.11111 − 7)/5.33333
```

```
    = 2.66667 − 0.02083    = 2.64583
x₃ = 2.64583 − (7.00043 − 7)/5.29167
    = 2.64583 − 0.00008    = 2.64575

True value √7 = 2.64575131…, so x₃ is already correct to 5 d.p.
Errors:  e₀ = 0.354,  e₁ = 0.021,  e₂ = 0.00008,  e₃ < 10⁻⁷.
Each step squares the previous error (quadratic convergence).
```

Worked Example 12 — Trapezium rule, 5 strips, for $\int_1^3 \sqrt{x}\,dx$

The exact value is $\left[\frac{2}{3}x^{3/2}\right]_1^3 = \frac{2}{3}(3\sqrt{3}-1) = 2\sqrt{3} - \frac{2}{3} \approx 2.79743$.

```
n = 5, a = 1, b = 3, h = (3 − 1)/5 = 0.4.
Tabulate y_i = √x_i:
   x₀ = 1.0  →  y₀ = 1.00000
   x₁ = 1.4  →  y₁ = 1.18322
   x₂ = 1.8  →  y₂ = 1.34164
   x₃ = 2.2  →  y₃ = 1.48324
   x₄ = 2.6  →  y₄ = 1.61245
   x₅ = 3.0  →  y₅ = 1.73205

Apply the formula:
   I ≈ (h/2) [y₀ + 2(y₁ + y₂ + y₃ + y₄) + y₅]
     = (0.4/2) [1.00000 + 2(1.18322 + 1.34164 + 1.48324 + 1.61245) + 1.73205]
     = 0.2 [1.00000 + 2(5.62055) + 1.73205]
     = 0.2 [1.00000 + 11.24110 + 1.73205]
     = 0.2 × 13.97315
     ≈ 2.79463.

Compare with exact 2.79743 − error ≈ 0.00280 (about 0.1 %).
√x is concave (f''(x) = −(1/4)x^(−3/2) < 0), so the rule UNDER-estimates ✓.
```

Worked Example 13 — Convergence check for $x_{n+1} = \cos(x_n)$ from $x_0 = 1$

```
Here g(x) = cos x, g'(x) = −sin x.
The fixed point α of cos satisfies α = cos α, with α ≈ 0.7391.
At α:  |g'(α)| = |sin(0.7391)| ≈ 0.6736 < 1.
Since |g'(α)| < 1, the iteration converges to α. ✓

Verify starting from x₀ = 1:
x₁ = cos 1       = 0.54030
x₂ = cos 0.54030 = 0.85755
x₃ = cos 0.85755 = 0.65429
x₄ = cos 0.65429 = 0.79348
x₅ = cos 0.79348 = 0.70137
x₆ = cos 0.70137 = 0.76396
 ...
x₃₀ ≈ 0.73909        (converging, alternating above/below α).

Convergence rate = 0.6736 per step — linear. It takes about 15 iterations
to reach 4 d.p. (since 0.6736¹⁵ ≈ 0.00002).
```

11. Formula Reference

Method	Formula	Notes
Change of sign	$f(a) \cdot f(b) < 0 \Rightarrow \exists\, \alpha \in (a, b)$	f continuous; existence only.
Bisection	$c = (a+b)/2$; retain half containing sign change	Width after n steps: $(b-a)/2^n$.
Fixed-point iteration	$x_{n+1} = g(x_n)$	Converges locally iff $\lvert g'(\alpha)\rvert < 1$.
Newton-Raphson	$x_{n+1} = x_n - \dfrac{f(x_n)}{f'(x_n)}$	Quadratic near simple root; fails if $f'(x_n) = 0$.
Trapezium rule	$\dfrac{h}{2}[y_0 + 2(y_1 + \cdots + y_{n-1}) + y_n]$	$h = (b-a)/n$; error $O(h^2)$.

12. Chapter Summary and Examiner's Checklist

Chapter-wide checklist for Numerical Methods.

- **Change of sign:** state continuity, quote $f(a)$ and $f(b)$ with signs, conclude root in (a, b).
- **Bisection:** tabulate $a_n, b_n, c_n, f(c_n)$ and the retained half at every step.
- **Fixed-point iteration:** before iterating, check $|g'(\alpha)| < 1$. After iterating, stop when $|x_{n+1} - x_n|$ is below tolerance.
- **Newton-Raphson:** write down $f'(x)$ and the iteration formula explicitly; beware $f'(x_n) = 0$ and poor starting values.
- **Trapezium rule:** state n strips and h; build a clean table of x_i, y_i; substitute carefully, with interior terms doubled.
- **Always quote the degree of accuracy** — "correct to 3 d.p." etc. — at the end.
- **Concavity check:** for the trapezium rule, decide over/under-estimate from the sign of f''.
- **Sketch the function** before choosing a starting value for any iterative method.
- **Verify** the final root by substituting back and checking $f(\alpha) \approx 0$.

Typical OCR examiner complaints (from mark reports).

- Omitting the statement "f is continuous" when applying the change-of-sign test.
- Stating that a sign change in $[a, b]$ means *exactly* one root, rather than *at least* one.
- In fixed-point iteration, failing to verify $|g'(\alpha)| < 1$, or giving only the final answer with no iteration table.
- Claiming a Newton-Raphson iteration "diverges because the function is not differentiable", when the real reason is $f'(x_0) = 0$ or a poor starting value.
- Confusing "number of strips" with "number of ordinates" in the trapezium rule — n strips produces $n + 1$ ordinates.
- Doubling the end ordinates y_0 and y_n (should be counted once) or failing to double the interior ordinates (should be counted twice).
- Rounding intermediate results to 3 d.p. and then quoting the final answer to 5 d.p. — rounding error propagates.

© RAG Learning · Not for redistribution

- Giving no explicit statement of accuracy ("to 3 d.p.") on the final line.

Formula snapshot (memorise).

- **Change of sign:** f continuous on $[a, b]$, $f(a)f(b) < 0 \Rightarrow$ at least one root in (a, b).
- **Bisection width:** $w_n = (b - a)/2^n$; midpoint error $\leq w_n/2$.
- **Fixed-point iteration:** $x_{n+1} = g(x_n)$; converges iff $|g'(\alpha)| < 1$.
- **Newton-Raphson:** $x_{n+1} = x_n - f(x_n)/f'(x_n)$; quadratic near a simple root.
- **Trapezium rule:** $\int_a^b f\,dx \approx \frac{h}{2}[y_0 + 2(y_1 + \cdots + y_{n-1}) + y_n]$, $h = (b - a)/n$.
- **Trapezium error:** $O(h^2)$; doubling n divides error by 4.
- **Concavity:** $f'' > 0 \Rightarrow$ over-estimate; $f'' < 0 \Rightarrow$ under-estimate.